Python Objects

A variable is a location in memory used to store some data (value).

We don't need to declare a variable before using it. In Python, we simply assign a value to a variable and it will exist. We don't even have to declare the type of the variable. This is handled internally according to the type of value we assign to the variable.

Iteration is a general term for taking each item of something, one after another. Any time you use a loop, explicit or implicit, to go over a group of items, that is iteration.

In Python, iterable and iterator have specific meanings.

An iterable is an object which returns an iterator, example types: list, string, tuple, dict, set

Immutable vs Immutable

Everything in Python is an Object, every variable holds an object instance. When an object is initiated, it is assigned a unique object id. Its type is defined at runtime and once set can never change, however its state can be changed if it is mutable. Simple put, a mutable object can be changed after it is created, and an immutable object can't.

Objects of built-in types like (int, float, bool, str, tuple, unicode) are immutable. Objects of built-in types like (list, set, dict) are mutable. Custom classes are generally mutable. To simulate immutability in a class, one should override attribute setting and deletion to raise exceptions.

Class	Description	Immutable?
bool	Boolean value	✓
int	integer (arbitrary magnitude)	✓
float	floating-point number	✓
list	mutable sequence of objects	
tuple	immutable sequence of objects	✓
str	character string	✓
set	unordered set of distinct objects	
frozenset	immutable form of set class	✓
dict	associative mapping (aka dictionary)	

Variable assignment

We use the assignment operator (=) to assign values to a variable. Any type of value can be assigned to any valid variable.

```
a = 5

b = 3.2

c = "Hello"
```

Here, we have three assignment statements. 5 is an integer assigned to the variable a.

Similarly, 3.2 is a floating point number and "Hello" is a string (sequence of characters) assigned to the variables b and c respectively.

Multiple assignments

In Python, multiple assignments can be made in a single statement as follows:

```
a, b, c = 5, 3.2, "Hello"
```

If we want to assign the same value to multiple variables at once, we can do this as

```
x = y = z = "same"
```

This assigns the "same" string to all the three variables.

Data types in Python

Every value in Python has a datatype. Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

Python Numbers

Integers, floating point numbers and complex numbers falls under Python numbers category. They are defined as int, float and complex class in Python.

```
a = 5

print(a, "is of type", type(a))

a = 2.0

print(a, "is of type", type(a))

a = 1+2j
```

```
print(a, "is complex number?", type(a))
```

Python Strings

String is sequence of Unicode characters. We can use single quotes or double quotes to represent strings. Multi-line strings can be denoted using triple quotes, ''' or """.

```
s = 'Hello world!'

# s[4] = 'o'

print("s[4] = ", s[4])
```

String Slices

The "slice" syntax is a handy way to refer to sub-parts of sequences -- typically strings and lists. The slice s[start:end] is the elements beginning at start and extending up to but not including end. Suppose we have

```
s = "Hello"
```

s[1:4] is 'ell' -- chars starting at index 1 and extending up to but not including index 4

s[1:] is 'ello' -- omitting either index defaults to the start or end of the string

s[:] is 'Hello' -- omitting both always gives us a copy of the whole thing (this is the pythonic way to copy a sequence like a string or list)

s[1:100] is 'ello' -- an index that is too big is truncated down to the string length

The standard zero-based index numbers give easy access to chars near the start of the string. As an alternative, Python uses negative numbers to give easy access to the chars at the end of the string: s[-1] is the last char 'o', s[-2] is 'l' the next-to-last char, and so on. Negative index numbers count back from the end of the string:

s[-1] is 'o' -- last char (1st from the end)

s[-4] is 'e' -- 4th from the end

s[:-3] is 'He' -- going up to but not including the last 3 chars.

s[-3:] is 'llo' -- starting with the 3rd char from the end and extending to the end of the string.

```
# s[6:11] = 'world'
print("s[6:11] = ", s[6:11])
```

Concatenation:

In order to merge two strings into a single object, you may use the “+” operator. When writing code, that would look like this:

```
str1 = “Hello”
str2 = “World”
X = str1 + str2
 Print(X)
HelloWorld
# Generates error
# Strings are immutable in Python
s[5] ='d'
```

Python String Methods

S.upper() & S.lower()

The .upper() and .lower() string methods are self-explanatory. Performing the .upper() method on a string converts all of the characters to uppercase, whereas the lower() method converts all of the characters to lowercase.

```
>>> s = “Whereof one cannot speak, thereof one must be silent.”
>>> s.upper()
'WHEREOF ONE CANNOT SPEAK, THEREOF ONE MUST BE SILENT.'
```

```
>>> s.lower()
'whereof one cannot speak, thereof one must be silent.'
```

S.count()

The .count() method adds up the number of times a character or sequence of characters appears in a string. For example:

```
>>> s = "That that is is that that is not is not is that it it is"
>>> s.count("t")
13
```

S.find()

We search for a specific character or characters in a string with the .find() method.

```
s = "On the other hand, you have different fingers."
>>> s. find("hand")
13
```

The results tell us that “hand” begins at the 13th position in the sequence.

But if we want to find the second “o” we need to specify a range.

```
>>> s.find("o", 8)
20
```

S.replace()

Let’s say we want to increase the value of a statement. We do so with the .replace() method. For example:

The syntax of replace() is:

S.replace(old, new [, count])

```
>>> s = "I intend to live forever, or die trying."
>>> s.replace("to", "three")
```

'I intend three live forever, or die trying.'

```
>>> s.replace("fore", "five")
```

'I intend to live fivever, or die trying.'

S.title() & S.istitle()

The title() method returns a title cased version of the string. Meaning, the first character of the each word is capitalized (if the first character is a letter).

```
text = 'My favorite number is 25.'
print(text.title()) # My Favorite Number Is 25
```

The istitle() returns True if the string is a titlecased string. If not, it returns False.

```
s = 'Python Is Good.'
print(s.istitle()) # True
```

S.join()

The join() is a string method which returns a string concatenated with the elements of an iterable

```
numList = ['1', '2', '3', '4']
seperator = ', '
print(seperator.join(numList))
numTuple = ('1', '2', '3', '4')
print(seperator.join(numTuple))
s1 = 'abc'
s2 = '123'
""" Each character of s2 is concatenated to the front of s1"""
print('s1.join(s2):', s1.join(s2))
""" Each character of s1 is concatenated to the front of s2"""
print('s2.join(s1):', s2.join(s1))
```

Output:

```
###############
1, 2, 3, 4
1, 2, 3, 4
s1.join(s2): 1abc2abc3
s2.join(s1): a123b123c
###############
```

S.index()

The index() method returns the index of a substring inside the string (if found). If the substring is not found, it raises an exception.

The syntax of index() method for string is:

str.index(sub[, start[, end]])

```
sentence = 'Python programming is fun.'


result = sentence.index('is fun')
print("Substring 'is fun':", result)
result = sentence.index('a', 0,6)
print("Substring 'Java':", result)
```

S.isdigit()

The isdigit() method returns True if all characters in a string are digits. If not, it returns False.

```
s = "28212"
print(s.isdigit()) # True
# contains alphabets and spaces
s = 11
print(s.isdigit()) # AttributeError
```

S.islower() & S.isupper()

The islower() method returns True if all alphabets in a string are lowercase alphabets. If the string contains at least one uppercase alphabet, it returns False.

```
s = 'this is good'

print(s.islower()) # True
```

S.strip() & S.lstrip() & S.rstrip()

The strip() method returns a copy of the string with both leading and trailing characters removed (based on the string argument passed).

```
 input = "    test string    "

print("Before strip-", input)

Before strip-     test string

input = input.strip()

print("After strip-", input)

After strip- test string
```

Case 2: Strip with args

```
input = "    test string    "

print("Before strip-", input)

Before strip- ##1##test string##1111

input = input.strip("1#")

print("After strip-", input)

After strip- test string
```

Python Lists

```
 colors = ['red', 'blue', 'green']
```

```
print colors[0]    ## red
```

List is an ordered sequence of items. It is one of the most used datatype in Python and is very flexible. All the items in a list do not need to be of the same type.

Declaring a list is pretty straight forward. Items separated by commas are enclosed within brackets [ ]

List Slices

Slices work on lists just as with strings, and can also be used to change sub-parts of the list.

```
list = ['a', 'b', 'c', 'd']
print list[1:-1]   ## ['b', 'c']
list[0:2] = 'z'    ## replace ['a', 'b'] with ['z']
print list         ## ['z', 'c', 'd']
```

Slicing: (can be used with any indexable iterable objects) - list, string, tuple

```
alist = [11,22,33,44,55,66,77,88,99,10]
print(alist[2:]) # from 2nd index till the end
print(alist[:3]) # from start to 2nd index
print(alist[:]) # from start to end
print(alist[1:6]) # from 1st index to 5th index
print(alist[1:8:2]) # from 1st index to 7th index with a gap of 2(skips ONE object between each) 1,3,5,7
print(alist[::3]) # from start to end, step by 3. 1,4,7,10
```

```
[33, 44, 55, 66, 77, 88, 99, 10]
[11, 22, 33]
[11, 22, 33, 44, 55, 66, 77, 88, 99, 10]
[22, 33, 44, 55, 66]
[22, 44, 66, 88]
[11, 44, 77, 10]
```

List Concatenation:

a = [1, 2, 3, 4]

b = [5, 6, 7, 8]

c = a + b

>>> c = [1, 2, 3, 4, 5, 6, 7, 8]

List Methods

list.append(elem)

adds a single element to the end of the list. Common error: does not return the new list, just modifies the original.

```
list = ['larry', 'curly', 'moe']
list.append('shemp')        ## append elem at end
print(list)
```

['larry', 'curly', 'moe', 'shemp']

list.insert(index, elem)

inserts the element at the given index, shifting elements to the right.

```
list = ['larry', 'curly', 'moe']
list.insert(1, 'person')
```

['larry', 'person', 'curly', 'moe']

list.extend(iterable)

adds the elements in iterable to the end of the list. Using + or += on a list is similar to using extend().

```
list = ['larry', 'curly', 'moe']
list.extend("ABC")
list.extend([11,22,33])
```

['larry', 'curly', 'moe', 'A', 'B', 'C', 11, 22, 33]

list.index(elem)

searches for the given element from the start of the list and returns its index. Throws a ValueError if the element does not appear (use "in" to check without a ValueError).

```
list = ['larry', 'curly', 'moe']

i = list.index("curly")
```

```
1
```

list.remove(elem)

searches for the first instance of the given element and removes it (throws ValueError if not present)

```
list = ['larry', 'curly', 'moe']

list.remove("curly")
```

```
['larry', 'moe']
```

list.sort()

sorts the list in place (does not return it). (The sorted() function shown below is preferred.)

```
list = ['larry', 'curly', 'moe']

list.sort()
```

```
['curly', 'larry', 'moe']
```

list.reverse()

reverses the list in place (does not return it)

```
list = ['larry', 'curly', 'moe']

list.reverse()
```

```
['moe', 'curly', 'larry']
```

list.pop(index)

removes and returns the element at the given index. Returns the rightmost element if index is omitted (roughly the opposite of append()).

Notice that these are *methods* on a list object, while len() is a function that takes the list (or string or whatever) as an argument.

```
list = ['larry', 'curly', 'moe']

list.pop(1)
```

['larry', 'moe']

List Build Up

One common pattern is to start a list a the empty list [], then use append() or extend() to add elements to it:

```
list = []        ## Start as the empty list

list.append('a')   ## Use append() to add elements

list.append('b')

print(list)
```

[‘a’, ‘b']

Python Tuple

Tuple is an ordered sequence of items same as list.The only difference is that tuples are immutable. Tuples once created cannot be modified.

Tuples are used to write-protect data and are usually faster than list as it cannot change dynamically.

```
t = (5,'program', 1)

# t[1] = 'program'

print("t[1] = ", t[1])

# t[0:3] = (5, 'program', 1)

print("t[0:3] = ", t[0:3])

# Generates error

# Tuples are immutable
```

```
t[0] = 10
```

Tuple Concatenation:

```
a = (1, 2, 3, 4)
b = (5, 6, 7, 8)
c = a + b
>>> c = (1, 2, 3, 4, 5, 6, 7, 8)
```

Python Tuple:

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = "a", "b", "c", "d"
```

Tuple Functions:

T.count()

The count() method returns the number of occurrences of an element in a tuple.

In simple terms, count() method searches the given element in a tuple and returns how many times the element has occurred in it.

```
# vowels tuple
vowels = ('a', 'e', 'i', 'o', 'i', 'o', 'e', 'i', 'u')
# count element 'i'
```

```
count = vowels.count('i')

# print count

print('The count of i is:', count)

# count element 'p'

count = vowels.count('p')

# print count

print('The count of p is:', count)
```

```
The count of i is: 3

The count of p is: 0
```

T.index()

The index() method searches an element in a tuple and returns its index.

In simple terms, index() method searches for the given element in a tuple and returns its position.

However, if the same element is present more than once, the first/smallest position is returned.

If no element is found, a ValueError exception is raised indicating the element is not found.

```
# vowels tuple

vowels = ('a', 'e', 'i', 'o', 'i', 'u')

# element 'e' is searched

index = vowels.index('e')

# index is printed

print('The index of e:', index)

# element 'i' is searched

index = vowels.index('i')

# only the first index of the element is printed
```

```
print('The index of i:', index)
```

```
The index of e: 1
The index of e: 2
```

Python Dictionary

Dictionary is an unordered collection of key-value pairs.

It is generally used when we have a huge amount of data. Dictionaries are optimized for retrieving data. We must know the key to retrieve the value.

In Python, dictionaries are defined within braces {} with each item being a pair in the form key:value. Key and value can be of any type.

```
>>> d = {1:'value','key':2}
>>> type(d)
```

```
<class 'dict'>
```

We use key to retrieve the respective value. But not the other way around.

```
d = {1:'value','key':2}
print(type(d))
print("d[1] = ", d[1]);
print("d['key'] = ", d['key']);
# Generates error since key does not exist
print("d[2] = ", d[2]);
```

```
## Can build up a dict by starting with the the empty dict {}
## and storing key/value pairs into the dict like this:
## dict[key] = value-for-that-key
dict = {}
dict['a'] = 'alpha'
dict['g'] = 'gamma'
```

```
dict['o'] = 'omega'
print(dict)
```

{'a': 'alpha', 'g': 'gamma', 'o': 'omega'}

Built-in Dictionary Methods

D.clear()

The method clear() removes all items from the dictionary.

```
dict = {'Name': 'Zara', 'Age': 7};
print "Start Len : %d" %  len(dict)
dict.clear()
print "End Len : %d" %  len(dict)
```

Start Len : 2

End Len : 0

D.copy()

The method copy() returns a shallow copy of the dictionary.

```
dict1 = {'Name': 'Zara', 'Age': 7};
dict2 = dict1.copy()
print "New Dictinary : %s" %  str(dict2)
```

New Dictinary : {'Age': 7, 'Name': 'Zara'}

D.get()

The method get() returns a value for the given key. If key is not available then returns default value None.

Following is the syntax for get() method –

```
dict.get(key, default=None)
dict = {'Name': 'Zabra', 'Age': 7}

print "Value : %s" %  dict.get('Age')
print "Value : %s" %  dict.get('Education', "Never")
```

```
## Value : 7
## Value : Never
```

D.items()

The method items() returns a list of dict's (key, value) tuple pairs

```
dict = {'Name': 'Zara', 'Age': 7}
print "Value : %s" %  dict.items()
```

When we run above program, it produces following result –

```
Value : [('Age', 7), ('Name', 'Zara')]
```

D.keys()

The method keys() returns a list of all the available keys in the dictionary.

```
dict = {'Name': 'Zara', 'Age': 7}
print "Value : %s" %  dict.keys()
```

When we run above program, it produces following result –

```
Value : ['Age', 'Name']
```

D.setdefault()

The method setdefault() is similar to get(), but will set dict[key]=default if key is not already in dict.

```
dict = {'Name': 'Zara', 'Age': 7}
print "Value : %s" %  dict.setdefault('Age', None)
print "Value : %s" %  dict.setdefault('Sex', None)
```

When we run above program, it produces following result –

```
Value : 7
Value : None
```

D.update()

The method update() adds dictionary dict2's key-values pairs in to dict. This function does not return anything.

```
dict = {'Name': 'Zara', 'Age': 7}
```

```
dict2 = {'Sex': 'female' }

dict.update(dict2)
print "Value : %s" %  dict
## Value : {'Age': 7, 'Name': 'Zara', 'Sex': 'female'}
```

D.values()

The method values() returns a list of all the values available in a given dictionary.

```
dict = {'Name': 'Zara', 'Age': 7}
print "Value : %s" %  dict.values()
## Value : [7, 'Zara']
```

D.fromkeys()

The method fromkeys() creates a new dictionary with keys from seq and values set to value.

```
seq = ('name', 'age', 'sex')
dict = dict.fromkeys(seq)
``

print "New Dictionary : %s" %  str(dict)
dict = dict.fromkeys(seq, 10)
print "New Dictionary : %s" %  str(dict)
## New Dictionary : {'age': None, 'name': None, 'sex': None}
## New Dictionary : {'age': 10, 'name': 10, 'sex': 10}
```

D.pop()

The syntax of pop() method is

D.pop(key[, default])

If key is found - removed/popped element from the dictionary

If key is not found - value specified as the second argument (default)

If key is not found and default argument is not specified - KeyError exception is raised

```
sales = { 'apple': 2, 'orange': 3, 'grapes': 4 }

element = sales.pop('apple')

print('The popped element is:', element)

print('The dictionary is:', sales)
```

```
The popped element is: 2

The dictionary is: {'orange': 3, 'grapes': 4}
```

D.clear()

The syntax of clear() method is

D.clear()

Used to clear the dictionary

```
sales = { 'apple': 2, 'orange': 3, 'grapes': 4 }

sales.clear()

print(sales)
```

output:

```
###############

{}
```

D.copy()

The syntax of copy() method is

D_copy = D.pop()

Used to copy the dictionary

```
sales = { 'apple': 2, 'orange': 3, 'grapes': 4 }

s = sales.copy()

print(s)

print(sales)
```

output:

```
{ 'apple': 2, 'orange': 3, 'grapes': 4 }
{ 'apple': 2, 'orange': 3, 'grapes': 4 }
```

Python common functions (part – 1)

del(object)

With del, we specify a list, dictionary or other collection. We pass an index or key we want to remove. On lists, we can remove slices (ranges of elements) at once.

```
############# List ############
values = [100, 200, 300, 400, 500, 600]
# Use del to remove by an index or range of indexes.
del values[2]
print(values) # [100, 200, 400, 500, 600]
elements = ["A", "B", "C", "D"]
# Remove first element.
del elements[:1]
print(elements) # ["B", "C", "D"]
elements = ["A", "B", "C", "D"]
# Remove two middle elements.
del elements[1:3]
print(elements) # ["A", "D"]
elements = ["A", "B", "C", "D"]
# Remove second element only.
del elements[1]
print(elements) # ["A", "C", "D"]
```

Output

```
['B', 'C', 'D']
['A', 'D']
['A', 'C', 'D']
```

```
######### dictionary ##########
colors = {"red" : 100, "blue" : 50, "purple" : 75}
# Delete the pair with a key of "red."
del colors["red"]
print(colors)
```

Output

```
{'blue': 50, 'purple': 75}
```

```
########### string/int/flot ####
location = "beach" # immutable types
# Cannot remove part of a string with del.
del location[1] ## update also not possible, TypeError
print(location)
del location ## this will delete
```

id(object)

Return the "identity" of an object. This is an integer (or long integer) which is guaranteed to be unique and constant for this object during its lifetime. Two objects with non-overlapping lifetimes may have the same id() value

```
print('id of 5 =',id(5)) ## id of 5 = 140472391630016
a = 5
print('id of a =',id(a)) ## id of a = 140472391630016
```

type()

If the single object argument is passed to type(), it returns type of the given object.

```
numberList = [1, 2]
print(type(numberList)) ## <class 'list'>
numberDict = {1: 'one', 2: 'two'}
print(type(numberDict)) ## <class 'dict'>
```

chr(i)

Return a string of one character whose ASCII code is the integer i. For example, chr(97) returns the string 'a'. This is the inverse of ord(). The argument must be in the range [0..255], inclusive

```
print(chr(97)); # ‘a’
```

ord(c)

Given a string of length one, return an integer representing the Unicode code point of the character when the argument is a unicode object. For example,

ord('a') returns the integer 97

pow(x, y)

pow functions performs x to the power

```
print(2 ** 2) #8, using exponent
print(pow(2, 2)) #8
```

len(object)

In Python though we mostly measure the lengths of strings, lists, collections

Strings. Len returns the number of characters in a string. It counts spaces, punctuation, all characters the same. We must be careful with taking the len of a None variable—this fails.

Note: We can find the length of any iterable using len function. We can not find the length of int, bool, float. We can convert those non-iterable to string format and find the length

```
# Has length of 3:
value = "cat"
print(len(value)) # 3

# Has length of 0:
value = ""
print(len(value))
# Causes TypeError:
value = None
print(len(value)) # error
# Get length of list with len.
elements = [1, 2, 3]
print(len(elements)) # 3
# Get length of tuple.
items = ("cat", "dog", "bird", "shark")
print(len(items)) # 4
# Length of dictionary (pair count).
lookup = {"cat" : 4, "centipede" : 100}
print(len(lookup)) # 3
```

max & min

Max, min. In a list, we do not need to search for the largest or smallest element. Instead we use max and min. The maximum or minimum value is returned.

With two arguments, max returns the larger of the two. And min returns the smaller. We can use max and min to compare two variables.

Syntax:

```
c = max(ITERABLE)
```

```
c = min(ITERABLE)
```

```
values = [-100, 1, 10, 1000]
# Find the max and min elements.
print(max(values))
print(min(values))
```

Output

```
1000
-100
```

Sum

Syntax:

```
S = sum(ITERABLE_INT)
```

A list contains the numbers 10, 20 and 5. Its sum is 35. We can a for-loop to iterate and add up the element values. Or we can use a simpler built-in method.

Two arguments. The sum() built-in receives two arguments. The first is an iterable collection like list—these elements are summed. The second argument is added to the sum.

```
values = [10, 20, 5]
# The sum of the 3 values is this number.
result = sum(values)
print(result)
# Use sum with two arguments.
# ... The second argument is added to the element sum.
result = sum(values, 1000)
print(result)
```

Output

```
35
```

Range

Sequences are everywhere. Our alphabet and our numbers are sequences. And in computers, sequences of bits take meanings—they form numbers and values.

For looping in Python, we often use an immutable sequence called range. This built-in method returns a sequence based on the numbers we pass it. Range receives a start, end and step.

range(stop)

stop: Number of integers (whole numbers) to generate, starting from zero. eg. range(3) == [0, 1, 2].

range([start], stop[, step])

start: Starting number of the sequence.

stop: Generate numbers up to, but not including this number.

step: Difference between each number in the sequence.

range(4, 10, 2); # returns [4, 6, 8]